# RAG Knowledge Assistant — Full Technical Plan

> NestJS Backend · Next.js Frontend · pgvector · Vercel AI SDK · Ollama (dev) / Groq (prod)

---

## 0. Objective

Build a production-quality Knowledge Assistant where users upload documents (PDF / TXT / MD), index them via a RAG pipeline, and query them with LLM-generated answers streamed in real time. Backend: NestJS (TypeScript). Frontend: Next.js (TypeScript). Vector store: pgvector.

This document is the single source of truth. Implement every section in order. Do not deviate from architecture decisions described here.

---

## 1. Repository Structure

```
knowledge-assistant/
├── backend/                    # NestJS application
│   ├── src/
│   │   ├── main.ts
│   │   ├── app.module.ts
│   │   ├── auth/
│   │   ├── documents/
│   │   ├── embeddings/
│   │   ├── vector-store/
│   │   ├── query/
│   │   └── common/
│   ├── test/
│   ├── Dockerfile
│   ├── .env.example
│   └── package.json
├── frontend/                   # Next.js application
│   ├── src/
│   │   ├── app/
│   │   ├── components/
│   │   ├── lib/
│   │   └── hooks/
│   ├── Dockerfile
│   └── package.json
├── docker-compose.yml          # local dev (postgres + pgvector + ollama +
backend + frontend)
├── docker-compose.prod.yml     # prod (no ollama)
└── README.md
```

## 2. Tech Stack — Locked

### Backend

| Concern | Package | Version |
|---|---|---|
| Framework | `@nestjs/core`, `@nestjs/common` | ^10 |
| ORM | `@nestjs/typeorm`, `typeorm` | ^10 |
| Database driver | `pg` | ^8 |
| pgvector TypeORM | `pgvector` | latest |
| Validation | `class-validator`, `class-transformer` | latest |
| Auth | `@nestjs/passport`, `passport-jwt`, `@nestjs/jwt` | latest |
| Password hashing | `bcrypt`, `@types/bcrypt` | latest |
| File upload | `@nestjs/platform-express`, `multer` | latest |
| PDF parsing | `pdf-parse`, `@types/pdf-parse` | latest |
| Markdown metadata | `gray-matter` | latest |
| AI SDK (LLM + embed) | `ai` (Vercel AI SDK) | ^4 |
| Ollama provider | `ollama-ai-provider` | latest |
| Groq provider | `@ai-sdk/groq` | latest |
| Rate limiting | `@nestjs/throttler` | latest |
| Logging | `winston`, `nest-winston` | latest |
| Config | `@nestjs/config` | latest |
| HTTP | `axios` | latest |

### Frontend

| Concern | Package |
| --- | --- |
| Framework | Next.js 14 (App Router) |
| Styling | Tailwind CSS |
| HTTP client | `axios` |
| SSE streaming | native `EventSource` API |
| Auth state | `zustand` |
| Forms | `react-hook-form` + `zod` |

---

## 3. Environment Variables

### backend/.env.example

```env
# App
PORT=3001
NODE_ENV=development

# Database
DATABASE_HOST=localhost
DATABASE_PORT=5432
DATABASE_NAME=knowledge_db
DATABASE_USER=postgres
DATABASE_PASSWORD=postgres

# JWT
JWT_SECRET=supersecretjwtkey
JWT_EXPIRES_IN=7d

# LLM Provider: "ollama" | "groq"
LLM_PROVIDER=ollama

# Ollama (local dev)
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_LLM_MODEL=llama3.2:3b
OLLAMA_EMBED_MODEL=nomic-embed-text

# Groq (production)
GROQ_API_KEY=
GROQ_LLM_MODEL=llama-3.1-70b-versatile

# Nomic hosted embeddings (production alternative)
NOMIC_API_KEY=

# File Upload
UPLOAD_MAX_SIZE_MB=10
UPLOAD_DEST=./uploads

# RAG Config
CHUNK_SIZE=512
CHUNK_OVERLAP=50
TOP_K_RESULTS=5
VECTOR_DIMENSIONS=768
```

---

## 4. Docker Compose (Local Dev)

yaml

```yaml
# docker-compose.yml
version: '3.8'

services:
  postgres:
    image: pgvector/pgvector:pg16
    container_name: knowledge_postgres
    environment:
      POSTGRES_DB: knowledge_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - '5432:5432'
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U postgres']
      interval: 5s
      timeout: 5s
      retries: 5

  ollama:
    image: ollama/ollama
    container_name: knowledge_ollama
    ports:
      - '11434:11434'
    volumes:
      - ollama_data:/root/.ollama
    entrypoint: >
      sh -c "ollama serve &
             sleep 5 &&
             ollama pull llama3.2:3b &&
             ollama pull nomic-embed-text &&
             wait"

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: knowledge_backend
    ports:
      - '3001:3001'
    env_file: ./backend/.env
    environment:
      LLM_PROVIDER: ollama
```

```yaml
      LLM_PROVIDER: ollama
      OLLAMA_BASE_URL: http://ollama:11434
      DATABASE_HOST: postgres
    depends_on:
      postgres:
        condition: service_healthy
      ollama:
        condition: service_started
    volumes:
      - ./backend/uploads:/app/uploads

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: knowledge_frontend
    ports:
      - '3000:3000'
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:3001

volumes:
  pgdata:
  ollama_data:
```

---

## 5. Database Schema

Enable pgvector extension first — do this in a TypeORM migration:

```sql
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm;  -- for BM25 / full-text search
```

### Entities

### User

```typescript
@Entity('users')
export class User {
  @PrimaryGeneratedColumn('uuid') id: string;
  @Column({ unique: true }) email: string;
  @Column() passwordHash: string;
  @Column({ type: 'enum', enum: UserRole, default: UserRole.USER }) role: UserRole;
  @CreateDateColumn() createdAt: Date;
  @UpdateDateColumn() updatedAt: Date;
  @OneToMany(() => Document, doc => doc.user) documents: Document[];
}

export enum UserRole { ADMIN = 'admin', USER = 'user' }
```

### Document

```typescript
@Entity('documents')
export class Document {
  @PrimaryGeneratedColumn('uuid') id: string;
  @Column() filename: string;
  @Column() originalName: string;
  @Column({ type: 'enum', enum: DocumentStatus, default: DocumentStatus.PENDING }) s
  @Column({ type: 'enum', enum: FileType }) fileType: FileType;
  @Column({ nullable: true }) pageCount: number;
  @Column({ type: 'jsonb', default: {} }) metadata: Record<string, any>;
  @CreateDateColumn() createdAt: Date;
  @ManyToOne(() => User, user => user.documents) user: User;
  @OneToMany(() => Chunk, chunk => chunk.document) chunks: Chunk[];
}

export enum DocumentStatus { PENDING = 'pending', PROCESSING = 'processing', READY =
export enum FileType { PDF = 'pdf', TXT = 'txt', MD = 'md' }
```

### Chunk

```typescript
@Entity('chunks')
export class Chunk {
  @PrimaryGeneratedColumn('uuid') id: string;
  @Column('text') content: string;
  @Column() chunkIndex: number;
  @Column() tokenCount: number;
  @Column({ type: 'jsonb', default: {} }) metadata: Record<string, any>;
  // tsvector column for BM25 full-text search — managed via raw migration
  // ALTER TABLE chunks ADD COLUMN tsv tsvector GENERATED ALWAYS AS (to_tsvector('en
  // CREATE INDEX chunks_tsv_idx ON chunks USING GIN(tsv);
  @ManyToOne(() => Document, doc => doc.chunks, { onDelete: 'CASCADE' }) document: D
  @OneToOne(() => Embedding, emb => emb.chunk) embedding: Embedding;
}
```

### Embedding

```typescript
@Entity('embeddings')
export class Embedding {
  @PrimaryGeneratedColumn('uuid') id: string;
  // pgvector column — managed via raw migration, NOT via TypeORM column decorator
  // ALTER TABLE embeddings ADD COLUMN vector vector(768);
  // CREATE INDEX embeddings_vector_idx ON embeddings USING ivfflat (vector vector_c
  @OneToOne(() => Chunk, chunk => chunk.embedding, { onDelete: 'CASCADE' })
  @JoinColumn()
  chunk: Chunk;
  @Column() chunkId: string;
}
```

**Note on pgvector columns:** TypeORM does not natively understand the `vector` type. Manage the `vector` column and its index exclusively via raw SQL migrations. Use the `pgvector` npm package for query helpers.

---

## 6. NestJS Module Breakdown

### 6.1 AppModule

```typescript
@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true, envFilePath: '.env' }),
    TypeOrmModule.forRootAsync({
      useFactory: (config: ConfigService) => ({
        type: 'postgres',
        host: config.get('DATABASE_HOST'),
        port: config.get<number>('DATABASE_PORT'),
        database: config.get('DATABASE_NAME'),
        username: config.get('DATABASE_USER'),
        password: config.get('DATABASE_PASSWORD'),
        entities: [User, Document, Chunk, Embedding],
        migrations: ['dist/migrations/*.js'],
        migrationsRun: true,
        synchronize: false,  // NEVER true in production
      }),
      inject: [ConfigService],
    }),
    WinstonModule.forRoot(winstonConfig),
    ThrottlerModule.forRoot([{ ttl: 60000, limit: 60 }]),
    AuthModule,
    DocumentsModule,
    EmbeddingsModule,
    VectorStoreModule,
    QueryModule,
  ],
})
export class AppModule {}
```

### 6.2 AuthModule

**Files:**

- `auth.module.ts`
- `auth.controller.ts` — POST `/auth/signup`, POST `/auth/login`
- `auth.service.ts` — signup, login, validateUser
- `jwt.strategy.ts` — PassportStrategy(Strategy)
- `jwt-auth.guard.ts` — extends AuthGuard('jwt')
- `roles.guard.ts` — checks `@Roles()` decorator against `req.user.role`
- `roles.decorator.ts` — `@Roles(UserRole.ADMIN)`
- `current-user.decorator.ts` — `@CurrentUser()` param decorator
- `dto/signup.dto.ts` — email, password (min 8 chars)
- `dto/login.dto.ts` — email, password

**Auth flow:**

1. Signup: hash password with bcrypt (rounds: 10), save user, return JWT
2. Login: find user by email, compare password, return JWT
3. JWT payload: `{ sub: userId, email, role }`
4. JWT strategy: extract Bearer token, validate, attach user to request

**DTOs use class-validator:**

```typescript
export class SignupDto {
  @IsEmail() email: string;
  @IsString() @MinLength(8) password: string;
}
```

### 6.3 DocumentsModule

**Files:**

- `documents.module.ts`
- `documents.controller.ts`
- `documents.service.ts`
- `pdf-parser.service.ts`
- `chunker.service.ts`
- `dto/upload-document.dto.ts`
- `dto/list-documents.dto.ts` — pagination

**Controller endpoints:**

```
POST   /documents/upload     — multipart/form-data, field: file
GET    /documents            — paginated list (page, limit, query params)
GET    /documents/:id        — single document with chunk count
DELETE /documents/:id        — delete document + cascade chunks + embeddings
```

All endpoints protected by `JwtAuthGuard`. Users only see their own documents. `DELETE` restricted to document owner or ADMIN role.

**Upload flow (documents.service.ts):**

```
1. Validate file type (pdf/txt/md only), size (max from config)
2. Save file to UPLOAD_DEST via multer
3. Create Document record with status PENDING
4. Trigger async processing (do NOT await — return document immediately)
5. Async: set status PROCESSING → parse → chunk → embed → set status READY
6. On error: set status FAILED, log error
```

**Pagination — GET /documents:**

```typescript
export class ListDocumentsDto {
  @IsOptional() @Type(() => Number) @IsInt() @Min(1) page: number = 1;
  @IsOptional() @Type(() => Number) @IsInt() @Min(1) @Max(100) limit: number = 10;
  @IsOptional() @IsString() search?: string;
}

// Response shape:
{
  data: Document[],
  meta: { page, limit, total, totalPages }
}
```

### 6.4 PdfParserService

```typescript
@Injectable()
export class PdfParserService {
  async parse(filePath: string, fileType: FileType): Promise<ParsedDocument> {
    switch (fileType) {
      case FileType.PDF:
        return this.parsePdf(filePath);
      case FileType.TXT:
        return this.parseTxt(filePath);
      case FileType.MD:
        return this.parseMd(filePath);
    }
  }

  private async parsePdf(filePath: string): Promise<ParsedDocument> {
    const buffer = fs.readFileSync(filePath);
    const data = await pdfParse(buffer);
    return {
      content: data.text.trim(),
      metadata: { pages: data.numpages, info: data.info },
    };
  }

  private parseTxt(filePath: string): ParsedDocument {
    const content = fs.readFileSync(filePath, 'utf-8');
    return { content: content.trim(), metadata: {} };
  }

  private parseMd(filePath: string): ParsedDocument {
    const raw = fs.readFileSync(filePath, 'utf-8');
    const { content, data: frontmatter } = matter(raw);
    return { content: content.trim(), metadata: frontmatter };
  }
}

interface ParsedDocument {
  content: string;
  metadata: Record<string, any>;
}
```

### 6.5 ChunkerService

Sentence-boundary chunking with overlap. Do NOT use a library — implement this directly.

typescript

```typescript
@Injectable()
export class ChunkerService {
  constructor(private config: ConfigService) {}

  chunk(text: string): ChunkResult[] {
    const size = this.config.get<number>('CHUNK_SIZE', 512);
    const overlap = this.config.get<number>('CHUNK_OVERLAP', 50);

    // Normalize whitespace
    const normalized = text.replace(/\s+/g, ' ').trim();

    // Split on sentence boundaries
    const sentences = normalized.match(/[^.!?\n]+[.!?\n]+|\S[^.!?\n]*/g) ?? [normali

    const chunks: ChunkResult[] = [];
    let current = '';
    let index = 0;

    for (const sentence of sentences) {
      if (current.length + sentence.length > size && current.length > 0) {
        chunks.push({
          content: current.trim(),
          chunkIndex: index++,
          tokenCount: current.trim().length,
        });
        // carry overlap
        const words = current.split(' ');
        const overlapWords = words.slice(-Math.floor(overlap / 5));
        current = overlapWords.join(' ') + ' ' + sentence;
      } else {
        current += (current ? ' ' : '') + sentence;
      }
    }

    if (current.trim()) {
      chunks.push({
        content: current.trim(),
        chunkIndex: index,
        tokenCount: current.trim().length,
      });
    }

    return chunks;
  }
}
```

```typescript
}

interface ChunkResult {
  content: string;
  chunkIndex: number;
  tokenCount: number;
}
```

### 6.6 EmbeddingsModule

Provider-agnostic embedding service using Vercel AI SDK.

```typescript
@Injectable()
export class EmbeddingsService {
  private model: EmbeddingModel<string>;

  constructor(private config: ConfigService) {
    const provider = config.get('LLM_PROVIDER');
    if (provider === 'ollama') {
      this.model = createOllama({
        baseURL: config.get('OLLAMA_BASE_URL') + '/api',
      }).embedding(config.get('OLLAMA_EMBED_MODEL', 'nomic-embed-text'));
    } else {
      // Groq does not have embeddings — use nomic hosted or openai
      // Default to nomic hosted for consistency with local model
      this.model = createOllama({ baseURL: 'https://api-atlas.nomic.ai/v1' })
        .embedding('nomic-embed-text-v1.5');
    }
  }

  async embedText(text: string): Promise<number[]> {
    const { embedding } = await embed({ model: this.model, value: text });
    return embedding;
  }

  async embedBatch(texts: string[]): Promise<number[][]> {
    const { embeddings } = await embedMany({ model: this.model, values: texts });
    return embeddings;
  }
}
```

**Note:** `embed` and `embedMany` are from the `ai` package (Vercel AI SDK).

### 6.7 VectorStoreModule

Raw SQL queries for pgvector operations. TypeORM's query builder does not support vector ops natively — use `DataSource.query()` for all vector operations.

typescript

```typescript
@Injectable()
export class PgVectorService {
  constructor(
    @InjectDataSource() private dataSource: DataSource,
  ) {}

  async storeEmbedding(chunkId: string, vector: number[]): Promise<void> {
    await this.dataSource.query(
      `INSERT INTO embeddings (id, "chunkId", vector)
       VALUES (gen_random_uuid(), $1, $2::vector)
       ON CONFLICT ("chunkId") DO UPDATE SET vector = $2::vector`,
      [chunkId, JSON.stringify(vector)],
    );
  }

  async similaritySearch(
    queryVector: number[],
    topK: number,
    documentIds?: string[],
  ): Promise<VectorSearchResult[]> {
    const filter = documentIds?.length
      ? `AND d.id = ANY($3::uuid[])`
      : '';
    const params: any[] = [JSON.stringify(queryVector), topK];
    if (documentIds?.length) params.push(documentIds);

    const rows = await this.dataSource.query(
      `SELECT
        c.id AS "chunkId",
        c.content,
        c."chunkIndex",
        c.metadata,
        d.id AS "documentId",
        d."originalName",
        1 - (e.vector <=> $1::vector) AS score
       FROM embeddings e
       JOIN chunks c ON c.id = e."chunkId"
       JOIN documents d ON d.id = c."documentId"
       WHERE 1=1 ${filter}
       ORDER BY e.vector <=> $1::vector
       LIMIT $2`,
      params,
    );
    return rows;
  }
```

```
  }

  async bm25Search(
    query: string,
    topK: number,
    documentIds?: string[],
  ): Promise<VectorSearchResult[]> {
    const filter = documentIds?.length ? `AND d.id = ANY($3::uuid[])` : '';
    const params: any[] = [query, topK];
    if (documentIds?.length) params.push(documentIds);

    const rows = await this.dataSource.query(
      `SELECT
        c.id AS "chunkId",
        c.content,
        c."chunkIndex",
        c.metadata,
        d.id AS "documentId",
        d."originalName",
        ts_rank(c.tsv, plainto_tsquery('english', $1)) AS score
       FROM chunks c
       JOIN documents d ON d.id = c."documentId"
       WHERE c.tsv @@ plainto_tsquery('english', $1) ${filter}
       ORDER BY score DESC
       LIMIT $2`,
      params,
    );
    return rows;
  }
}

interface VectorSearchResult {
  chunkId: string;
  content: string;
  chunkIndex: number;
  metadata: Record<string, any>;
  documentId: string;
  originalName: string;
  score: number;
}
```

### 6.8 QueryModule

**Files:**

- `query.module.ts`
- `query.controller.ts`
- `query.service.ts`
- `retrieval.service.ts` — hybrid search + RRF reranking
- `dto/query.dto.ts`

**Controller endpoints:**

```
POST /query          — non-streaming, returns full answer
GET  /query/stream   — SSE streaming response
```

Both endpoints: `JwtAuthGuard` + `@Throttle({ default: { limit: 10, ttl: 60000 } })`

**Query DTO:**

```typescript
export class QueryDto {
  @IsString() @MinLength(3) @MaxLength(1000) query: string;
  @IsOptional() @IsArray() @IsUUID('4', { each: true }) documentIds?: string[];
}
```

**RetrievalService — Hybrid Search with RRF:**

Reciprocal Rank Fusion merges vector and BM25 results. RRF score formula: `score = Σ 1 / (k + rank)` where k=60 (standard constant).

typescript

```typescript
@Injectable()
export class RetrievalService {
  constructor(
    private vectorStore: PgVectorService,
    private embeddings: EmbeddingsService,
    private config: ConfigService,
  ) {}

  async retrieve(query: string, documentIds?: string[]): Promise<RetrievedChunk[]> {
    const topK = this.config.get<number>('TOP_K_RESULTS', 5);

    // Run vector and BM25 search in parallel
    const [vectorResults, bm25Results] = await Promise.all([
      this.vectorSearchWithQuery(query, topK * 2, documentIds),
      this.vectorStore.bm25Search(query, topK * 2, documentIds),
    ]);

    // RRF merge
    return this.reciprocalRankFusion(vectorResults, bm25Results, topK);
  }

  private async vectorSearchWithQuery(
    query: string,
    topK: number,
    documentIds?: string[],
  ) {
    const vector = await this.embeddings.embedText(query);
    return this.vectorStore.similaritySearch(vector, topK, documentIds);
  }

  private reciprocalRankFusion(
    listA: VectorSearchResult[],
    listB: VectorSearchResult[],
    topK: number,
    k = 60,
  ): RetrievedChunk[] {
    const scores = new Map<string, { chunk: VectorSearchResult; score: number }>();

    const addList = (list: VectorSearchResult[]) => {
      list.forEach((item, rank) => {
        const existing = scores.get(item.chunkId);
        const rrfScore = 1 / (k + rank + 1);
        if (existing) {
          existing.score += rrfScore;
        } else {
```

```
      } else {
        scores.set(item.chunkId, { chunk: item, score: rrfScore });
      }
    });
  };

  addList(listA);
  addList(listB);

  return Array.from(scores.values())
    .sort((a, b) => b.score - a.score)
    .slice(0, topK)
    .map(({ chunk, score }) => ({ ...chunk, rrfScore: score }));
  }
}
```

**QueryService — LLM with Streaming:**

typescript

```typescript
@Injectable()
export class QueryService {
  constructor(
    private retrieval: RetrievalService,
    private config: ConfigService,
  ) {}

  private getModel(): LanguageModel {
    return this.config.get('LLM_PROVIDER') === 'ollama'
      ? createOllama({ baseURL: this.config.get('OLLAMA_BASE_URL') + '/v1' })(
          this.config.get('OLLAMA_LLM_MODEL', 'llama3.2:3b'),
        )
      : createGroq({ apiKey: this.config.get('GROQ_API_KEY') })(
          this.config.get('GROQ_LLM_MODEL', 'llama-3.1-70b-versatile'),
        );
  }

  private buildSystemPrompt(chunks: RetrievedChunk[]): string {
    const context = chunks
      .map((c, i) => `[${i + 1}] Source: ${c.originalName}\n${c.content}`)
      .join('\n\n---\n\n');

    return `You are a knowledge assistant. Answer the user's question using ONLY the
If the answer is not in the context, say "I don't have enough information to answer
Do not make up information. Always reference which source you used.

CONTEXT:
${context}`;
  }

  async *streamAnswer(
    query: string,
    documentIds?: string[],
  ): AsyncGenerator<string> {
    const chunks = await this.retrieval.retrieve(query, documentIds);

    if (!chunks.length) {
      yield 'No relevant documents found for your query.';
      return;
    }

    const { textStream } = await streamText({
      model: this.getModel(),
      system: this.buildSystemPrompt(chunks),
      messages: [{ role: 'user', content: query }]
```

```
    messages: [{ role: 'user', content: query }],
  });

  for await (const delta of textStream) {
    yield delta;
  }
}

async getAnswer(query: string, documentIds?: string[]): Promise<QueryResponse> {
  const chunks = await this.retrieval.retrieve(query, documentIds);

  if (!chunks.length) {
    return { answer: 'No relevant documents found.', sources: [] };
  }

  const { text } = await generateText({
    model: this.getModel(),
    system: this.buildSystemPrompt(chunks),
    messages: [{ role: 'user', content: query }],
  });

  return {
    answer: text,
    sources: chunks.map(c => ({
      chunkId: c.chunkId,
      documentId: c.documentId,
      documentName: c.originalName,
      chunkIndex: c.chunkIndex,
      score: c.rrfScore,
    })),
  };
}
}
```

**QueryController — SSE Streaming:**

```typescript
@Controller('query')
@UseGuards(JwtAuthGuard)
export class QueryController {
  constructor(private queryService: QueryService) {}

  @Post()
  @Throttle({ default: { limit: 10, ttl: 60000 } })
  async query(@Body() dto: QueryDto, @CurrentUser() user: User) {
    return this.queryService.getAnswer(dto.query, dto.documentIds);
  }

  @Sse('stream')
  @Throttle({ default: { limit: 10, ttl: 60000 } })
  streamQuery(
    @Query() dto: QueryDto,
    @CurrentUser() user: User,
  ): Observable<MessageEvent> {
    return new Observable(subscriber => {
      (async () => {
        try {
          for await (const chunk of this.queryService.streamAnswer(
            dto.query,
            dto.documentIds,
          )) {
            subscriber.next({ data: { chunk, done: false } } as MessageEvent);
          }
          subscriber.next({ data: { chunk: '', done: true } } as MessageEvent);
          subscriber.complete();
        } catch (err) {
          subscriber.error(err);
        }
      })();
    });
  }
}
```

---

## 7. Common Layer

### GlobalExceptionFilter

Catch all unhandled exceptions. Log them via Winston. Return consistent error shape:

```json
{
  "statusCode": 500,
  "message": "Internal server error",
  "timestamp": "2025-01-01T00:00:00Z",
  "path": "/api/query"
}
```

```typescript
@Catch()
export class GlobalExceptionFilter implements ExceptionFilter {
  constructor(@Inject(WINSTON_MODULE_PROVIDER) private logger: Logger) {}

  catch(exception: unknown, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse<Response>();
    const request = ctx.getRequest<Request>();

    const status =
      exception instanceof HttpException
        ? exception.getStatus()
        : HttpStatus.INTERNAL_SERVER_ERROR;

    const message =
      exception instanceof HttpException
        ? exception.message
        : 'Internal server error';

    this.logger.error('Unhandled exception', {
      status,
      message,
      path: request.url,
      stack: exception instanceof Error ? exception.stack : undefined,
    });

    response.status(status).json({
      statusCode: status,
      message,
      timestamp: new Date().toISOString(),
      path: request.url,
    });
  }
}
```

### LoggingInterceptor

Log every request/response with duration:

```typescript
@Injectable()
export class LoggingInterceptor implements NestInterceptor {
  constructor(@Inject(WINSTON_MODULE_PROVIDER) private logger: Logger) {}

  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    const req = context.switchToHttp().getRequest();
    const { method, url } = req;
    const start = Date.now();

    return next.handle().pipe(
      tap(() => {
        this.logger.info(`${method} ${url} ${Date.now() - start}ms`);
      }),
    );
  }
}
```

### Winston Config

typescript

```typescript
export const winstonConfig = {
  transports: [
    new winston.transports.Console({
      format: winston.format.combine(
        winston.format.timestamp(),
        winston.format.colorize(),
        winston.format.printf(({ timestamp, level, message, ...meta }) =>
          `${timestamp} [${level}] ${message} ${Object.keys(meta).length ? JSON.stri
        ),
      ),
    }),
    new winston.transports.File({
      filename: 'logs/error.log',
      level: 'error',
      format: winston.format.combine(winston.format.timestamp(), winston.format.json
    }),
    new winston.transports.File({
      filename: 'logs/combined.log',
      format: winston.format.combine(winston.format.timestamp(), winston.format.json
    }),
  ],
};
```

### main.ts Bootstrap

```typescript
async function bootstrap() {
  const app = await NestFactory.create(AppModule, {
    logger: WinstonModule.createLogger(winstonConfig),
  });

  // Global prefix
  app.setGlobalPrefix('api');

  // Validation pipe — global
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,           // strip unknown properties
      forbidNonWhitelisted: true,
      transform: true,           // auto-transform types
      transformOptions: { enableImplicitConversion: true },
    }),
  );

  // Global exception filter
  app.useGlobalFilters(new GlobalExceptionFilter(app.get(WINSTON_MODULE_PROVIDER)));

  // Global logging interceptor
  app.useGlobalInterceptors(new LoggingInterceptor(app.get(WINSTON_MODULE_PROVIDER)

  // CORS
  app.enableCors({
    origin: process.env.FRONTEND_URL ?? 'http://localhost:3000',
    credentials: true,
  });

  await app.listen(process.env.PORT ?? 3001);
}
bootstrap();
```

---

## 8. Database Migrations

Use TypeORM migrations — never `synchronize: true` in production.

**Migration execution order:**

1. `CreateUsersTable`
2. `CreateDocumentsTable`
3. `CreateChunksTable` — includes tsvector column + GIN index
4. `CreateEmbeddingsTable` — includes vector column + ivfflat index
5. `EnableExtensions` — pgvector + pg_trgm

**Critical raw SQL in migrations:**

```typescript
// In CreateEmbeddingsTable migration
public async up(queryRunner: QueryRunner): Promise<void> {
  await queryRunner.query(`CREATE EXTENSION IF NOT EXISTS vector`);
  await queryRunner.query(`CREATE EXTENSION IF NOT EXISTS pg_trgm`);

  await queryRunner.query(`
    CREATE TABLE embeddings (
      id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
      "chunkId" UUID NOT NULL UNIQUE REFERENCES chunks(id) ON DELETE CASCADE,
      vector vector(768),
      "createdAt" TIMESTAMPTZ DEFAULT now()
    )
  `);

  await queryRunner.query(`
    CREATE INDEX embeddings_vector_idx
    ON embeddings USING ivfflat (vector vector_cosine_ops)
    WITH (lists = 100)
  `);
}

// In CreateChunksTable migration — add tsvector AFTER table creation
public async up(queryRunner: QueryRunner): Promise<void> {
  // ... create table first ...
  await queryRunner.query(`
    ALTER TABLE chunks
    ADD COLUMN tsv tsvector
    GENERATED ALWAYS AS (to_tsvector('english', content)) STORED
  `);
  await queryRunner.query(`
    CREATE INDEX chunks_tsv_idx ON chunks USING GIN(tsv)
  `);
}
```

## 9. Frontend (Next.js)

### Pages

```
/                   → redirect to /login if not authed, else /documents
/login              → login form
/register           → signup form
/documents          → paginated document list + upload button
/documents/[id]     → document detail + chunk count + status
/query              → query interface with SSE streaming
```

### Auth Flow

Store JWT in `httpOnly` cookie via a Next.js API route proxy, OR store in memory via zustand (simpler for interview scope). Use zustand for auth state:

```typescript
interface AuthStore {
  token: string | null;
  user: { id: string; email: string; role: string } | null;
  setAuth: (token: string, user: AuthStore['user']) => void;
  logout: () => void;
}
```

### SSE Streaming Hook

```typescript
export function useQueryStream() {
  const [answer, setAnswer] = useState('');
  const [isStreaming, setIsStreaming] = useState(false);
  const [sources, setSources] = useState<Source[]>([]);

  const stream = useCallback(async (query: string, documentIds?: string[]) => {
    setAnswer('');
    setIsStreaming(true);

    const params = new URLSearchParams({ query });
    if (documentIds?.length) {
      documentIds.forEach(id => params.append('documentIds', id));
    }

    const es = new EventSource(
      `${process.env.NEXT_PUBLIC_API_URL}/api/query/stream?${params}`,
      { withCredentials: true },
    );

    es.onmessage = (e) => {
      const data = JSON.parse(e.data);
      if (data.done) {
        setIsStreaming(false);
        es.close();
      } else {
        setAnswer(prev => prev + data.chunk);
      }
    };

    es.onerror = () => {
      setIsStreaming(false);
      es.close();
    };
  }, []);

  return { answer, isStreaming, stream, sources };
}
```

**Note:** EventSource does not support custom headers (cannot send Bearer token). Solution: use a short-lived query token, or proxy through Next.js API route which attaches the Authorization header server-side before forwarding to NestJS SSE endpoint.

**Next.js proxy route for SSE:**

```typescript
// app/api/query/stream/route.ts
export async function GET(req: NextRequest) {
  const token = req.cookies.get('access_token')?.value;
  const searchParams = req.nextUrl.searchParams.toString();

  const backendRes = await fetch(
    `${process.env.API_URL}/api/query/stream?${searchParams}`,
    { headers: { Authorization: `Bearer ${token}` } },
  );

  return new Response(backendRes.body, {
    headers: {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      Connection: 'keep-alive',
    },
  });
}
```

### UI Components Required

- `DocumentUpload` — drag-and-drop + file picker, shows upload progress + processing status
- `DocumentList` — paginated table with status badges (pending/processing/ready/failed)
- `DocumentFilter` — multi-select checkboxes for filtering query by document
- `QueryInput` — textarea + submit, shows streaming answer token by token
- `SourceCitations` — accordion list of source chunks with document name + score
- `AuthGuard` — HOC that redirects to /login if no token

---

## 10. Testing Plan

### Unit Tests (Jest)

Required coverage for evaluator:

```
auth.service.spec.ts
  ✓ signup creates user with hashed password
  ✓ signup throws ConflictException if email exists
  ✓ login returns JWT on valid credentials
  ✓ login throws UnauthorizedException on wrong password

documents.service.spec.ts
  ✓ findAll returns paginated results for user
  ✓ findAll does not return other users' documents
  ✓ remove throws ForbiddenException if not owner

chunker.service.spec.ts
  ✓ chunks text into correct number of segments
  ✓ respects chunk size limit
  ✓ includes overlap between consecutive chunks
  ✓ handles empty string input
  ✓ handles single short sentence

pdf-parser.service.spec.ts
  ✓ parses PDF buffer and returns text
  ✓ parses markdown and extracts frontmatter as metadata
  ✓ parses txt file correctly

retrieval.service.spec.ts
  ✓ calls both vector and BM25 search in parallel
  ✓ RRF correctly ranks items appearing in both lists higher
  ✓ deduplicates chunks appearing in both result sets
  ✓ respects topK limit

query.service.spec.ts
  ✓ returns no-results message when retrieval returns empty
  ✓ builds system prompt with context from retrieved chunks
```

**E2E Tests (Jest + Supertest)**

```
auth.e2e-spec.ts
  ✓ POST /api/auth/signup creates user and returns token
  ✓ POST /api/auth/login returns token
  ✓ protected routes reject requests without token

documents.e2e-spec.ts
  ✓ POST /api/documents/upload accepts PDF, returns document
  ✓ GET /api/documents returns paginated list
  ✓ GET /api/documents/:id returns document by id
  ✓ DELETE /api/documents/:id deletes document

query.e2e-spec.ts
  ✓ POST /api/query returns answer with sources
  ✓ GET /api/query/stream returns SSE events
  ✓ rate limiter returns 429 after limit exceeded
```

---

## 11. RBAC Implementation

```typescript
export enum UserRole { ADMIN = 'admin', USER = 'user' }

// roles.decorator.ts
export const Roles = (...roles: UserRole[]) => SetMetadata(ROLES_KEY, roles);

// roles.guard.ts
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector.getAllAndOverride<UserRole[]>(ROLES_KEY, [
      context.getHandler(),
      context.getClass(),
    ]);
    if (!requiredRoles) return true;
    const { user } = context.switchToHttp().getRequest();
    return requiredRoles.includes(user.role);
  }
}

// Usage on controller:
@Delete(':id')
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles(UserRole.ADMIN)
async adminDelete(@Param('id') id: string) { ... }
```

---

## 12. README Structure

The README must include these sections in order:

1. **Overview** — what the app does, one paragraph
2. **Architecture diagram** — text-based or mermaid diagram showing modules
3. **Tech stack** — table format
4. **Prerequisites** — Node 18+, Docker, Docker Compose, (Ollama optional if using Docker)
5. **Quick Start (Docker)** — `docker compose up` steps
6. **Quick Start (Local)** — step by step without Docker
7. **Environment variables** — table of all vars with descriptions and defaults
8. **API Reference** — all endpoints, method, path, auth required, request/response shape
9. **RAG Pipeline explanation** — how ingestion and retrieval work, why RRF
10. **Running tests** — `npm run test`, `npm run test:e2e`, `npm run test:cov`
11. **Production deployment** — env changes needed, docker-compose.prod.yml usage
12. **Design decisions** — why pgvector over Pinecone, why Vercel AI SDK, why custom chunker

---

## 13. Implementation Order

Implement in this exact order. Each step should be a git commit.

```
1.  Project scaffold (NestJS CLI + Next.js CLI)
2.  Docker Compose + postgres + ollama
3.  TypeORM setup + all entities
4.  Database migrations (run: npm run migration:run)
5.  ConfigModule + environment validation
6.  Winston logging setup
7.  Auth module (User entity + JWT + Guards + Decorators)
8.  Auth e2e tests passing
9.  DocumentsModule (upload endpoint + multer)
10. PdfParserService + ChunkerService
11. ChunkerService unit tests passing
12. EmbeddingsService (Vercel AI SDK + Ollama)
13. VectorStoreModule (pgvector + BM25 raw queries)
14. Documents async processing pipeline (parse → chunk → embed → store)
15. RetrievalService (hybrid search + RRF)
16. QueryService (streamText + generateText)
17. QueryController (POST /query + SSE /query/stream)
18. GlobalExceptionFilter + LoggingInterceptor wired
19. ThrottlerModule on query endpoints
20. All unit tests passing
21. All e2e tests passing
22. Next.js project scaffold + Tailwind
23. Auth pages (login + register)
24. Documents page (list + upload)
25. Query page (SSE streaming + source citations)
26. README complete
27. docker-compose.prod.yml
28. Final review pass
```

---

## 14. Key Constraints for Code Quality

- `synchronize: false` in TypeORM always — use migrations
- Never expose `passwordHash` in any response — use `@Exclude()` with ClassSerializerInterceptor
- All controllers use `@UseGuards(JwtAuthGuard)` — no public endpoints except auth routes
- All file uploads validated: mimetype whitelist + size limit — reject at controller level
- Async document processing errors must not crash the app — catch and update document status to FAILED
- SSE endpoint must close EventSource on client disconnect — use `request.on('close', ...)`
- Vector dimensions must match between embed model and pgvector column — `vector(768)` for nomic
- Chunk content must be sanitized before embedding — strip null bytes, normalize unicode
- All environment variables accessed via `ConfigService` — never `process.env` directly in services
- Rate limiting on query endpoints — 10 requests per minute per user
- Pagination required on GET /documents — never return unbounded lists